FundGoodDeeds / RELEASE 2

Project Design Document

TEAM 3C JPY

Jun Zheng <jz7259@rit.edu>

Evan Ke <ek6747@rit.edu>

Kevin Zheng <kz3175@rit.edu>

Google Doc link to the document with version history:

Release 1: SWEN-383-DesignDoc-3C.docx

Release 2: FundGoodDeeds / RELEASE 2

Table Of Contents

Table Of Contents 2
1 Project Summary 4
2 Design Overview 4
Process 4
Discover 4
Requirements 4
Constraints 4
Guidelines 5
Class Diagrams 6
Version 1 6
Version 2 6
Sequence Diagrams 8
Subsystem Diagrams 9
Version 1 9
Version 2 10
Development 11
Delivery 11
Challenges Faced 11
What We Learned & Successes 11
3 Subsystem Structure 13
4 Subsystems 14
4.1 Default Subsystem 14
4.2 Model Subsystem 15
4.3 View Subsystem 19
4.4 Controller Subsystem 23
5 Sequence Diagrams 25
5.1 Sequence Diagram #1 – Creating need 25
Purpose 25
Scenario and Feature 25
Process Explanation 25
5.2 Sequence Diagram #2 – Read Needs + Update 27
Purpose 27
Scenario and Feature 27
Process Explanation 27
5.3 Sequence Diagram #3 – Fulfilled Entry 29
Purpose 29
Scenario and Feature 29
Process Explanation 29
5.4 Sequence Diagram #4 – User Login (New Account) 31

Purpose 31
Scenario and Feature 31
Process Explanation 31
5.5 Sequence Diagram #5 – User Login (Existing Account) 33
Purpose 33
Scenario and Feature 33
Process Explanation 33
6 Pattern Usage 35
6.1 Pattern #1 - Composite Pattern 35
6.2 Pattern #2 - MVC Pattern 35
6.3 Pattern #3 - Observer Pattern 36
7 Rationale 37

1 Project Summary

In version 2.0, the financial resilience tracking program is further expanded beyond the basic implementation of phase 1.0. Users can now define funding sources, record income, update the threshold and available funds, and manage entries for selected dates.

There are three major additions to this phase: a new date-based tracking functionality, a new GUI utilizing Swing, and a user login system.

This project will build upon the first project, utilizing existing patterns such as MVC, composite, and observer patterns. The model will be expanded to include the new business requirements and functionality, and the graphical interface will be updated from text-based CLI to a Swing-based graphical interface.

Overall, phase 2 takes what we made in phase 1 and goes granular; it gives users a stronger view of their spending habits, incoming, and financial progress.

2 Design Overview

Process

For this second phase, we followed a similar process to phase 1

Discover -> Define -> Development -> Delivery -> Repeat based on feedback

Discover

Requirements

First, we had to figure out what the new requirements were. This includes managing sourcing funds, enforcing file format rules, daily logging with thresholds, edits, and deletions on per-day entries, new UI, etc.

Constraints

Secondly, we had to figure out project constraints. What were the parts of the assignments we *had* to follow? Upon inspection, we discovered that:

- ☑ We must build on the existing Version 1.0 design. We will be expanding on the program, not rewriting it.
- ☑ Continued usage of the MVC and Composite patterns.
- ☑ Continued usage of a CSV-based storage system for needs, funding, and logging.
- ☑ The saved needs.csv must follow the “no forward reference” and “no circular reference” rules for bundles.
- ☑ The program will have new date rules, including the one-week rule and restrictions on editing past thresholds/funds.
- ☑ We must integrate a Swing-based GUI unless explicitly approved otherwise by the instructor.

The design must reflect and incorporate all new Version 2.0 functional requirements (funding sources, thresholds, cascading defaults, deletions, editing flows, etc.).
- ☑ The team must adhere to the project timeline and deliverables for Version 2.0.

Guidelines

With the constraints given to us, we created guidelines that we had to follow throughout the project. This ensured that we met project requirements in a timely manner. These included:

- ☑ An extension rather than a rewriting of the second phase.
- ☑ Continued maintenance of SOC utilizing MVC
- ☑ Continued usage of CSVs
- ☑ New components such as the FundingManager, FundingSource, and new entries.
- ☑ Continued usage of the observer pattern
- ☑ We will implement a new Swing-based UI.
- ☑ Building upon the existing class and sequence diagrams for the new functionality.

Metrics of Success

Once this was done, we figured out what we considered metrics of success. What would be considered a “successful project”? Upon discussion, we concluded that in order for our project to be successful, we have to:

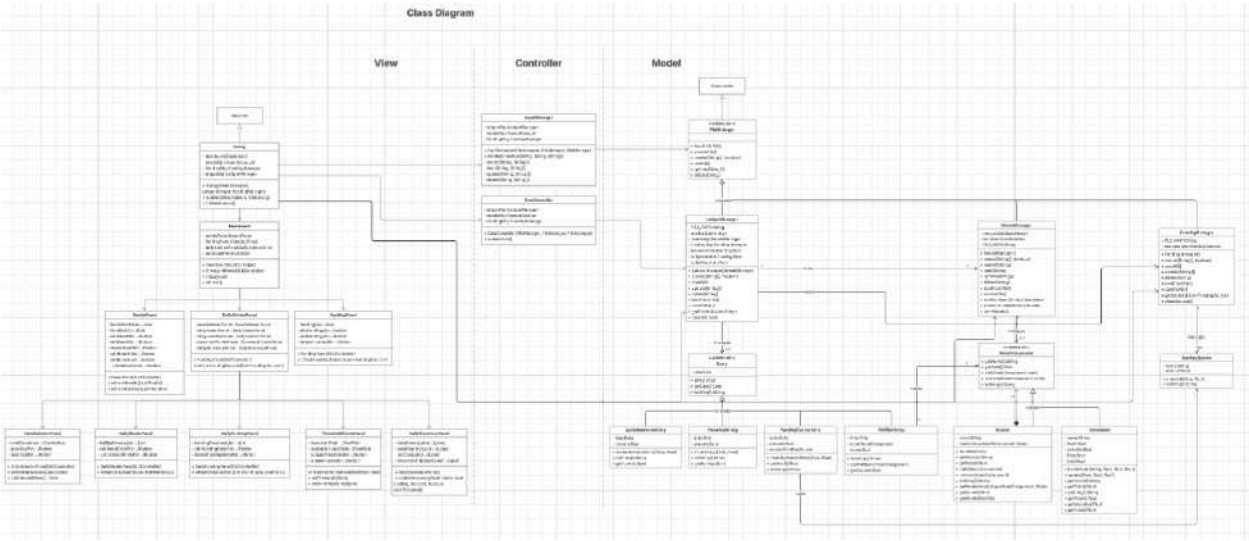
- ☑ Follow all project requirements
- ☑ Complete everything before the deliverable due dates
- ☑ Be prepared to talk about the parts we did for the project
- ☑ Learn and utilize the design patterns we learned in class
- ☑ Proper extension of the initial project

Based on this, we now know: The system now includes funding sources, daily thresholds, available funds, and a detailed daily log containing multiple types of entries. We will completely replace the view in the application with a Swing interface.

Define

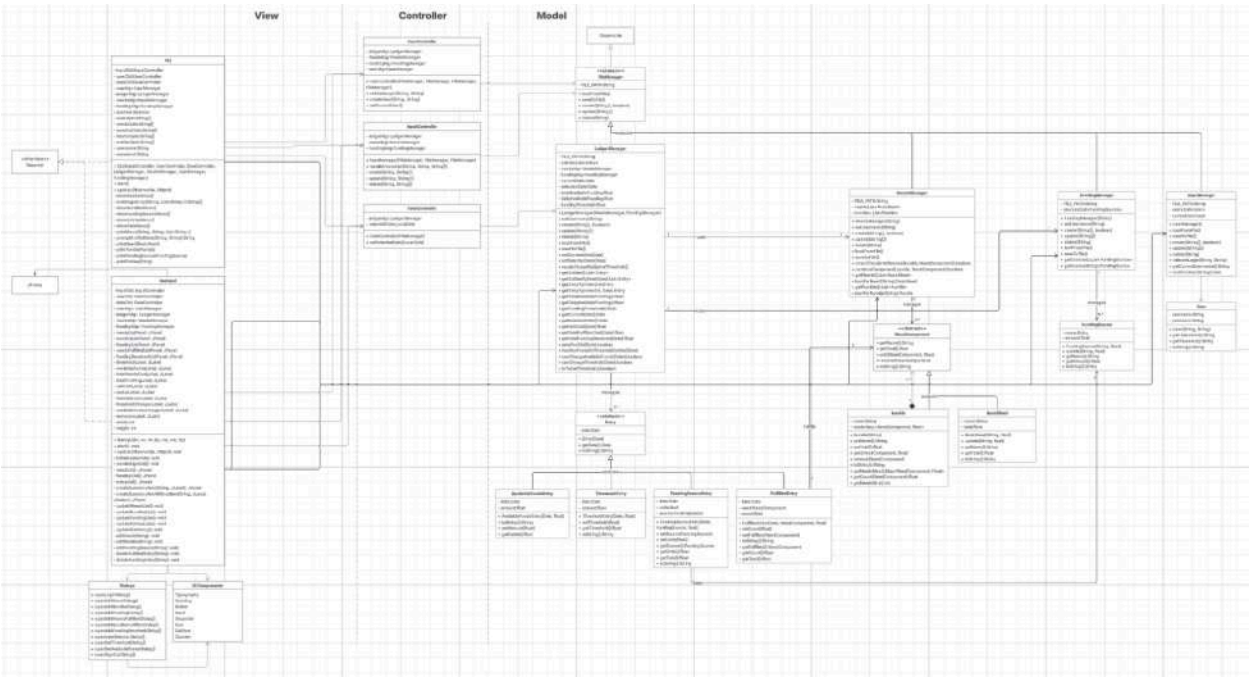
Class Diagrams

Version 1



This was our initial attempt to adapt the Release 1 design to the new requirements introduced in Release 2. In this version, we added the DateController, FundingManager, FundingSource, and SwingUI classes to support the new functionality required for the updated release.

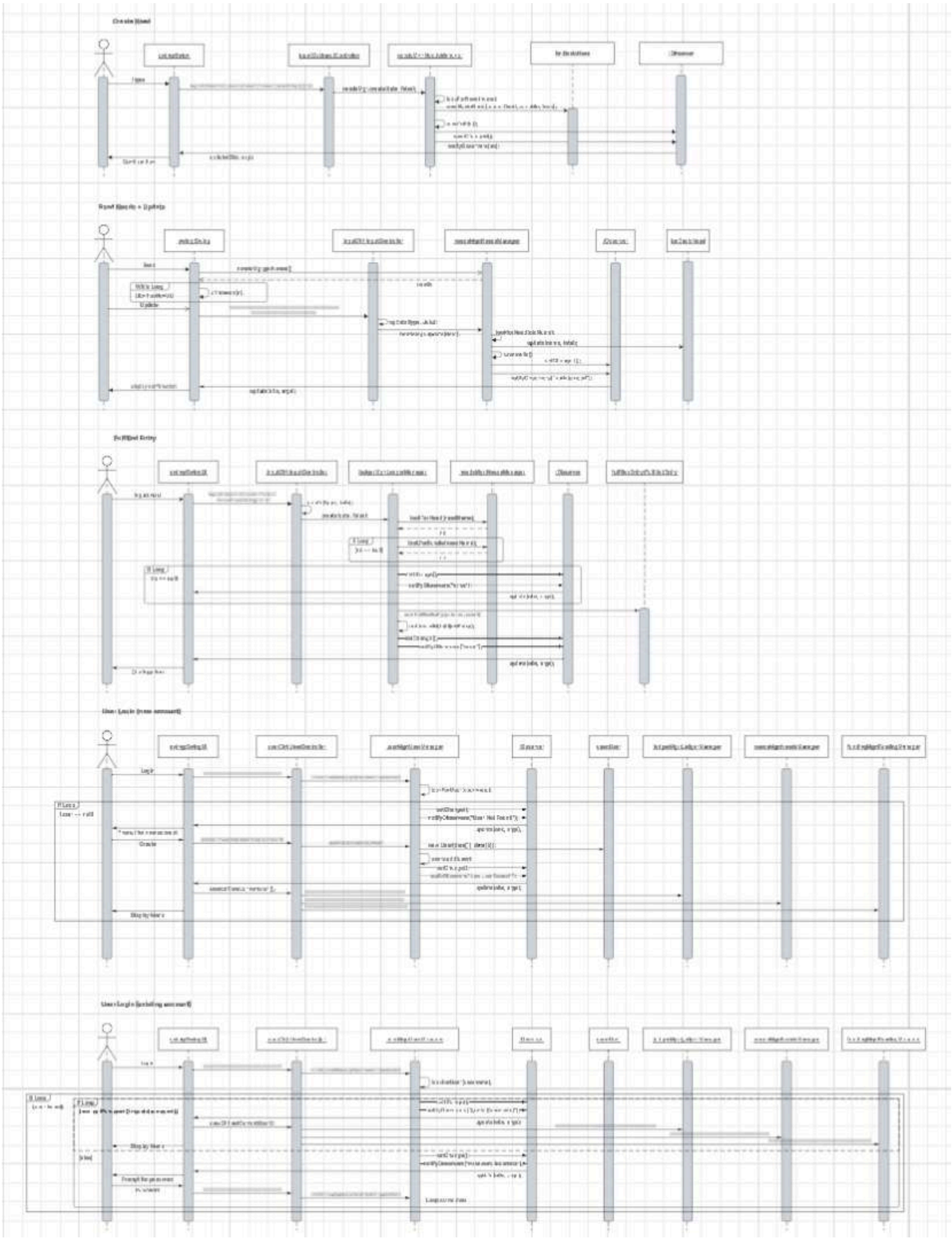
Version 2



After continuing implementation and incorporating the new user login system requirement, we expanded the diagram further by adding the UserManager and User classes. Additionally, the UI architecture was refactored to introduce reusable UI component classes, improving both maintainability and reusability across the application.

Sequence Diagrams

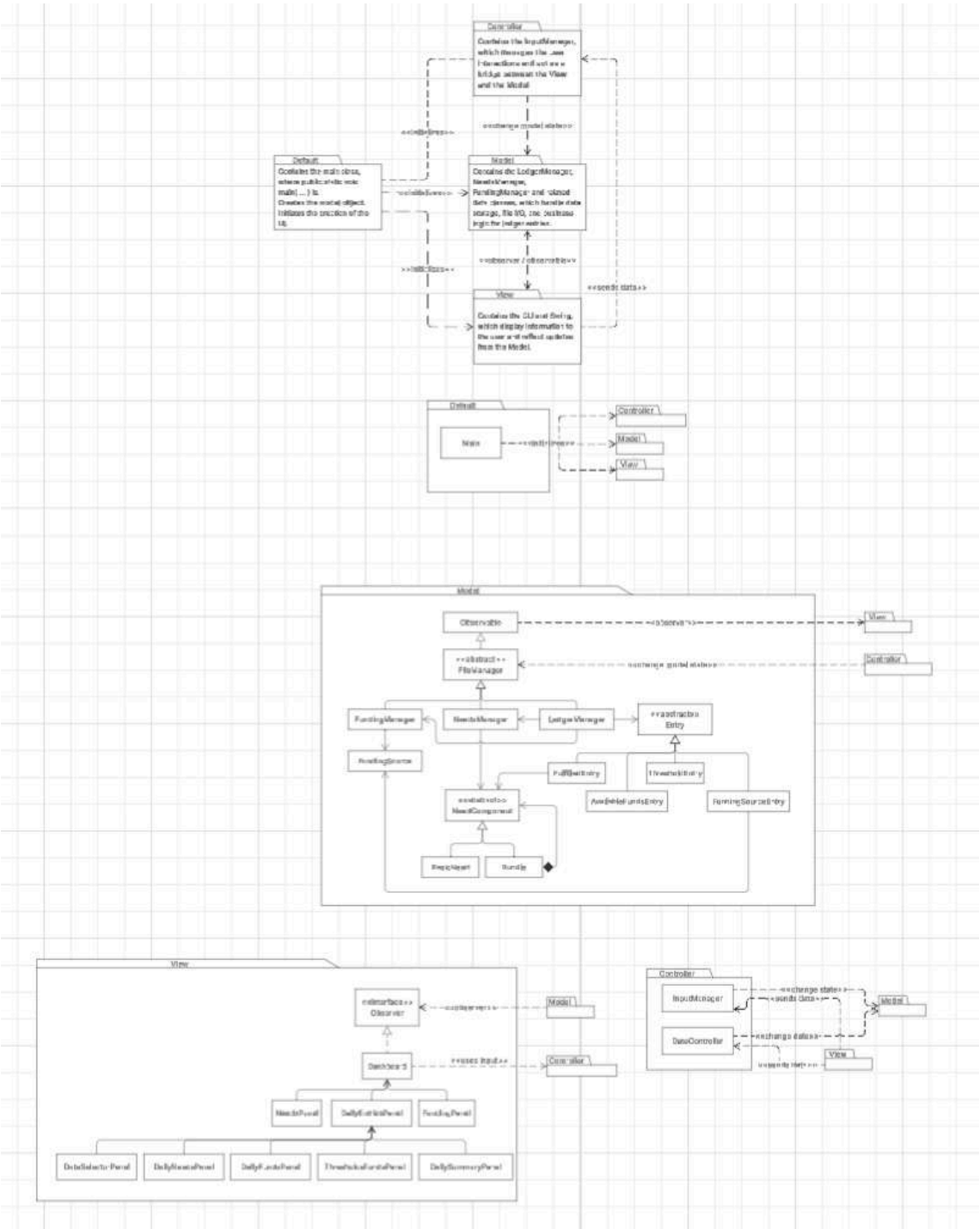
To learn more about our sequence diagrams, *reference our sequence diagram chapter*.



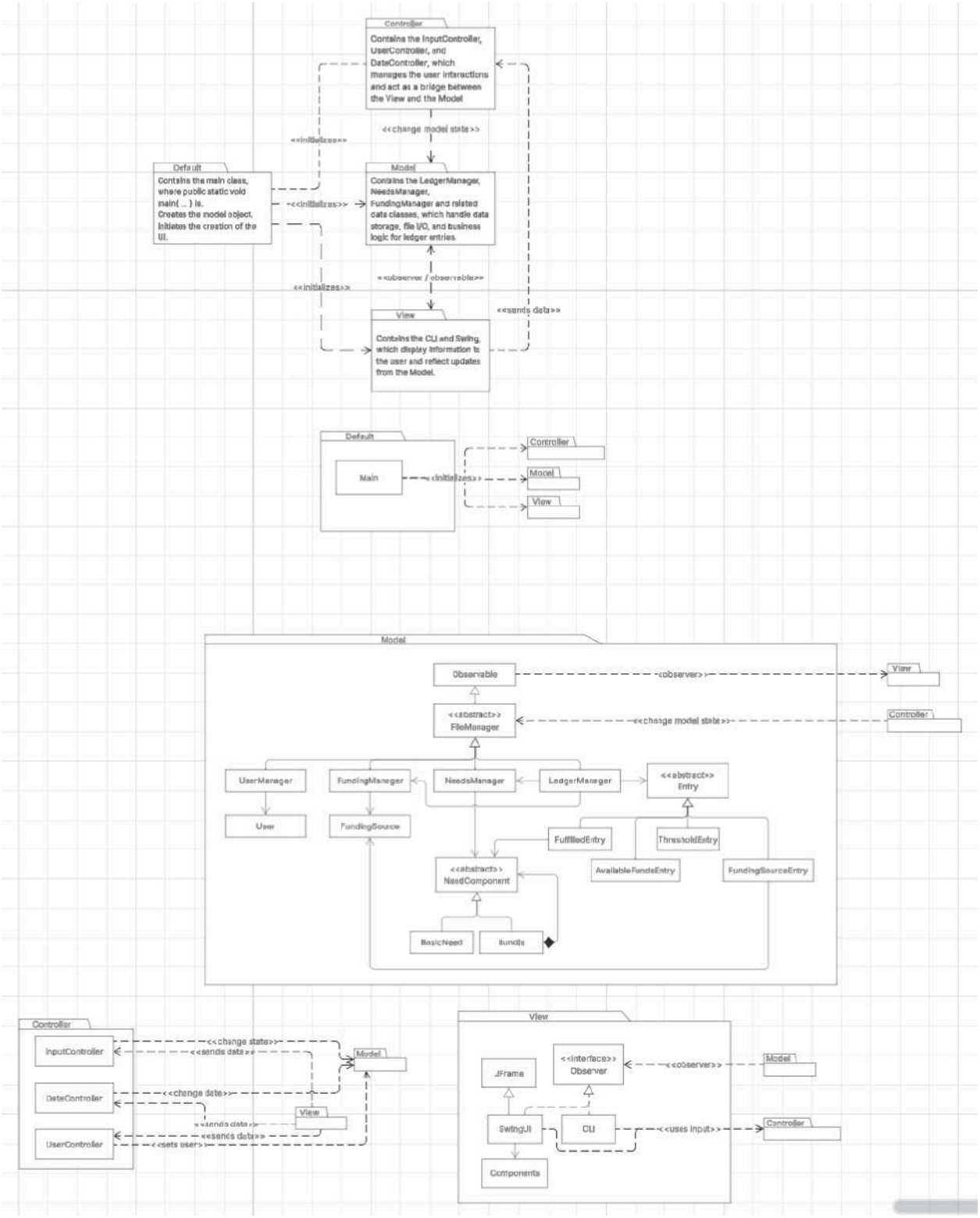
Subsystem Diagrams

The subsystem diagram was made to figure out how each class interacted and connected.
To learn more about our subsystem diagrams, *reference our subsystems chapter*.

Version 1



Version 2


Controller
Contains the InputController, UserController, and DateController, which manages the user interactions and act as a bridge between the View and the Model
<<change model state>>
<<initializes>>
Default
Contains the main class, where public static void main(...) is. Creates the model object, initiates the creation of the UI.
<<initializes>>
Model
Contains the LedgerManager, NeedsManager, FundingManager and related data classes, which handle data storage, file I/O, and business logic for ledger entries.
<<observer / observable>>
<<initializes>>
View
Contains the CLI and Swing, which display information to the user and reflect updates from the Model.
<<sends data>>
Default
Main
<<initializes>>
Controller
Model
View
Model
Observable
<<observer>>
View
<<abstract>>
FileManager
<<change model state>>
Controller
UserManager
FundingManager
NeedsManager
LedgerManager
<<abstract>>
Entry
User
FundingSource
FulfilledEntry
ThresholdEntry
AvailableFundsEntry
FundingSourceEntry
<<abstract>>
NeedComponent
BasicNeed
Bundle
Controller
InputController
<<change state>>
<<sends data>>
Model
DateController
<<change date>>
View
<<sends data>>
UserController
<<sends data>>
<<sets user>>
View
JFrame
<<interface>>
Observer
<<observer>>
Model
SwingUI
CLI
<<uses input>>
Controller
Components

Development

Once the diagrams were all completed, we moved on to development. We did our project in the 383 GitHub repo, and divided the work accordingly:

- ☑ Evan and Junheng work on Swing implementation
- ☑ Kevin works on the FundingManager and FundingSource
- ☑ Junheng works on the Available funds entry
- ☑ Evan works on Threshold entry, funding source entry
- ☑ Junheng works on the presentation
- ☑ Everyone works on the design doc together

To test the project yourself, you can *access the GitHub repository here.*

Delivery

Once development was done, we split delivery into three stages to ensure we met what we considered our metric of success:

1. Presentation: A presentation to talk about our process, work, and challenges. **You can view the presentation here (*Release 1*, *Release 2*).**
2. Revision: Based on feedback, we went back to revise and iterate on our work. Some of the design processes have been modified and adjusted based on the feedback.
3. Ensure that we properly expand upon the first phase.
4. Submission: Submission to the phase 2 Dropbox.

Challenges Faced

- Collaboration in a refactoring environment can be confusing and requires a lot of coordination
- Being open about schedule conflicts ensures proper communication
- Applying user-centered interaction to our interface.

What We Learned & Successes

By facing these challenges, we learned many new lessons in our process of achieving a successful project.

Lesson One

Compared to starting a project from scratch, collaboration in an already established environment can be hard to navigate. Proper communication ensures that spaghetti code does not happen.

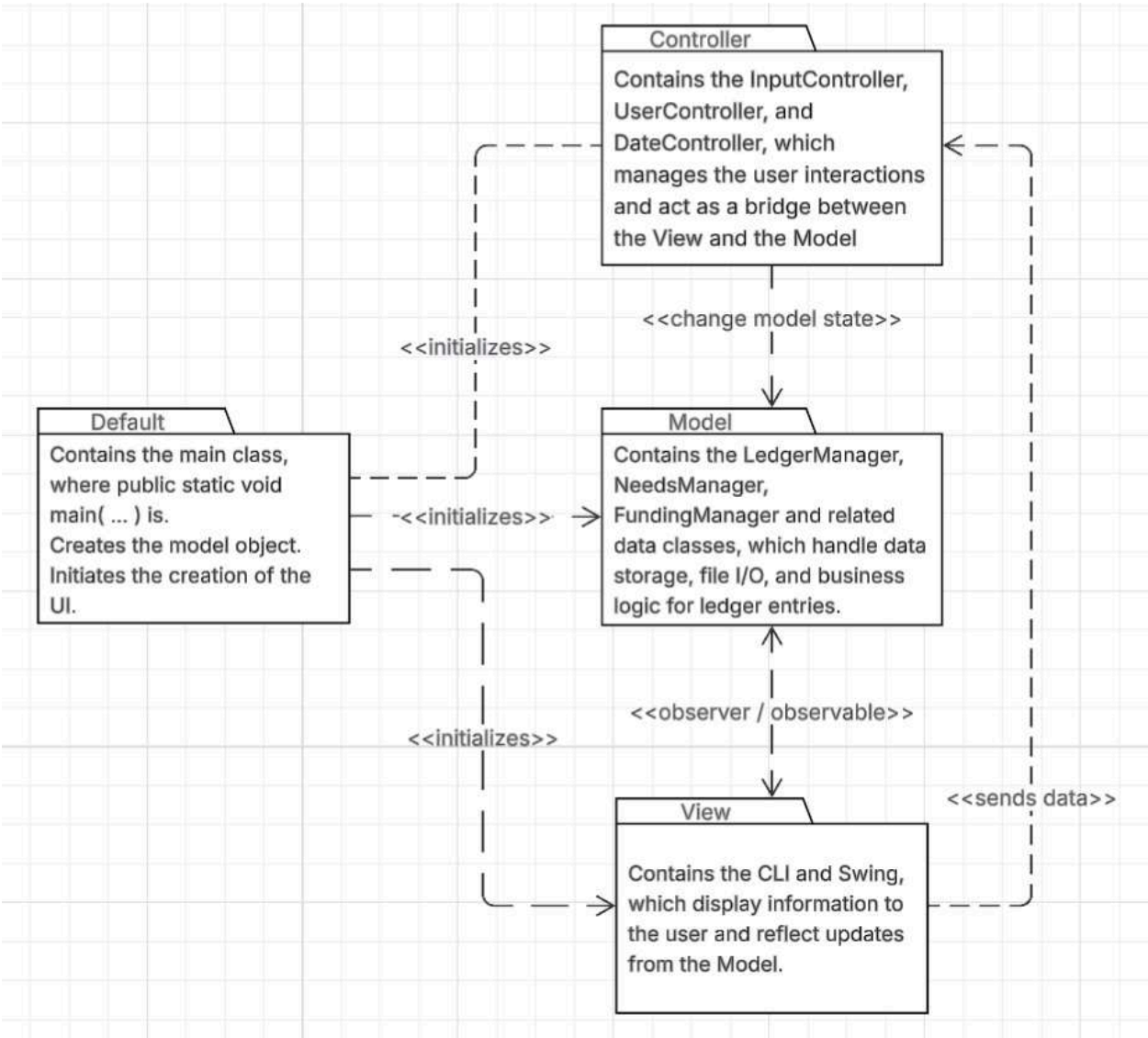
Lesson Two

Schedule conflicts such as traveling back for holidays, the actual holidays, exams, etc. Proper communication and having strong asynchronous communication allowed us to get the work done accordingly.

Lesson Three

Previously, we did not have a proper UI (CLI). Changing to a proper interface made us realize that design principles applied outside of backend code, and proper architecture had to also be applied to UI/UX and the frontend.

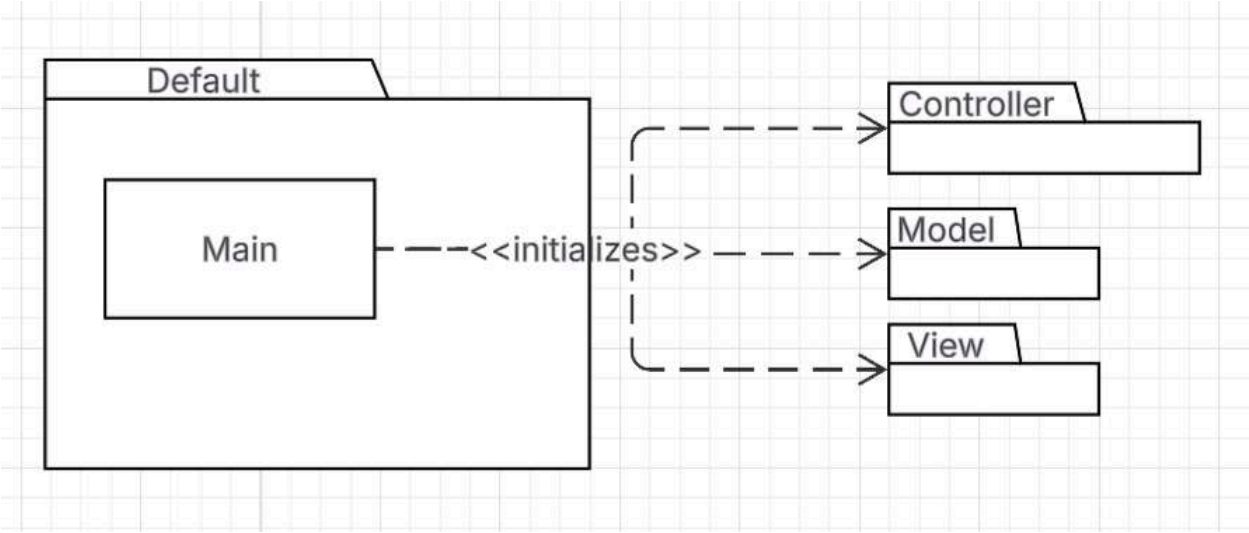
3 Subsystem Structure



4 Subsystems

4.1 Default Subsystem

Class Main	
Responsibilities	Create the Model object(s). Create the Controller object. Create the UI objects. Connect the subsystems together. Start the UI objects.
Collaborators (uses)	**Model.LedgerManager** - the class responsible for maintaining the entries. **Model.NeedsManager** - the class responsible for maintaining the needs and bundles. **Model.FundingManager** - the class responsible for maintaining the entries. **Model.UserManager** - the class responsible for maintaining the entries. **Controller.InputController** - the class responsible for changing model states. **Controller.DateController** - the class responsible for changing model states. **Controller.UserController** - the class responsible for changing model states. **View.CLI** - the primary view class. **View.SwingUI** - the primary view class.



4.2 Model Subsystem

FileManager, LedgerManager, Entry, AvailableFundsEntry, FulfilledEntry, ThresholdEntry, FundingSourceEntry, FundingManager, FundingSource, NeedsManager, NeedComponent, BasicNeed, Bundle, UserManager, User.

Class FileManager (abstract)	
Responsibilities	Define abstract methods for managing the storage of data. Provide a common interface for child classes to implement file operations. Handle saving and loading of entries, needs, or other objects in a structured format (CSV). Extends from the observable in order to be observed.
Collaborators (inherits)	**java.util.Observable** - So file managers can be observed by the View.

Class LedgerManager	
Responsibilities	Manage all ledger-related operations, including maintaining a collection of Entry objects. Interact with the NeedsManager to check if a need or bundle exists. Interact with the FundingManager to check if the funding source exists. Notify its observers when a request is completed/failed.
Collaborators (inherits)	**Model.FileManager** - Base operations for file management.
Collaborators (manages)	**Model.Entry** - Provide an abstraction layer for all entry objects.
Collaborators (interacts)	**Model.NeedsManager** - Handle storing and retrieving needs and bundles from files. **Model.FundingManager** - Handle storing and retrieving funding sources from files.
Collaborators (uses)	**java.sql.Date** - Hold the current and selected date.

Class Entry (abstract)	
Responsibilities	Provide a generic class for all entries in the ledger. Provide the name and date of the entry.
Collaborators (uses)	**java.sql.Date** - Hold the entry date.

Class AvailableFundsEntry	
Responsibilities	Represent an entry recording of the daily available funds.

Collaborators (inherits)	**Model.Entry** - Provide an abstraction layer for all entry objects.

Class FulfilledEntry	
Responsibilities	Represent a record of a fulfilled need. Provide the specific NeedComponent fulfilled and the amount fulfilled.
Collaborators (inherits)	**Model.Entry** - Provide an abstraction layer for all entry objects.
Collaborators (uses)	**Model.NeedComponent** - Define a base class for all need components.

Class ThresholdEntry	
Responsibilities	Represent an entry recording of the daily threshold.
Collaborators (inherits)	**Model.Entry** - Provide an abstraction layer for all entry objects.

Class FundingSourceEntry	
Responsibilities	Represent an entry recording of the amount of funding added to the total available funding.
Collaborators (inherits)	**Model.Entry** - Provide an abstraction layer for all entry objects.
Collaborators (uses)	**Model.FundingSource** - Provide information about a funding source.

Class FundingManager	
Responsibilities	Manage all funding source operations, including maintaining a collection of FundingSource objects. Notify its observers when a request is completed/failed.
Collaborators (inherits)	**Model.FileManager** - Base operations for file management.
Collaborators (uses)	**Model.FundingSource** - Provide information about a funding source.

Class FundingSource	
Responsibilities	Record and provide basic information about a funding source.

Class UserManager	
Responsibilities	Manage all user operations, including maintaining a collection of User objects. Notify its observers when a request is completed/failed.
Collaborators (inherits)	**Model.FileManager** - Base operations for file management.
Collaborators (uses)	**Model.User** - Provide information about a user.

Class User	
Responsibilities	Record and provide login information about a user.

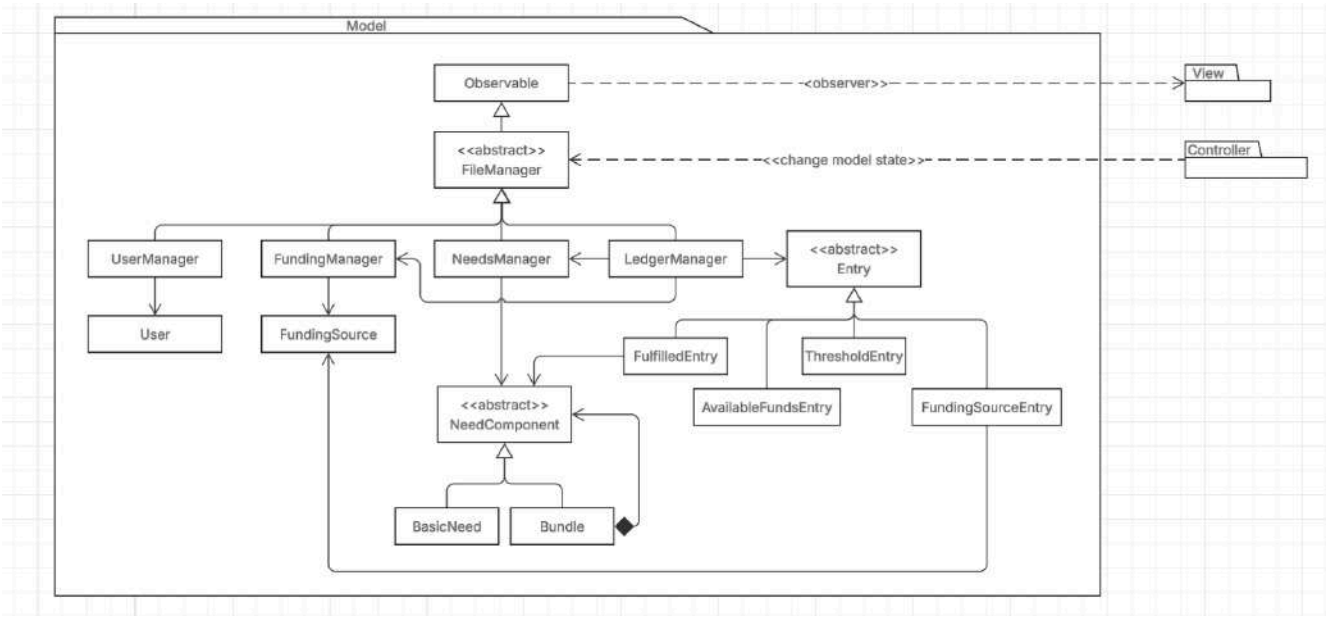
Class NeedsManager	
Responsibilities	Manage all needs-related operations, including maintaining a collection of BasicNeed and Bundle objects. Notify its observers when a request is completed.
Collaborators (inherits)	**Model.FileManager** - Base operations for file management.
Collaborators (uses)	**Model.NeedComponent** - Define a base class for all need components.

Class NeedComponent (abstract)	
Responsibilities	Provide a base class for the need and bundle. Provide name and date.

Class BasicNeed	
Responsibilities	Represent a single, fundamental need with associated costs. Provide name, total, fixed, variable, and fees.
Collaborators (implements)	**Model.NeedComponent** - Define a base class for all need components.

Class Bundle	
Responsibilities	Represent a group of needs bundled together. Provide name and list of NeedComponent objects. Add a new NeedComponent. Remove existing NeedComponent.

Collaborators (implements/composes)	**Model.NeedComponent** - Define a base class for all need components.



4.3 View Subsystem

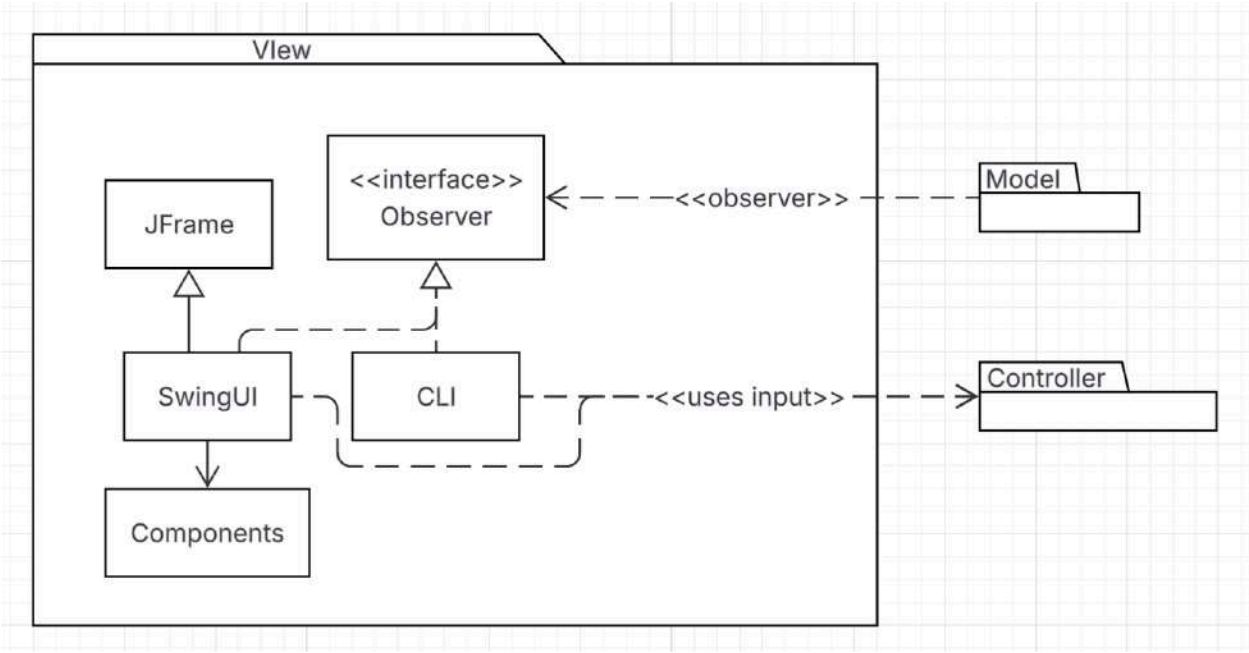
Class CLI	
Responsibilities	Route user commands to InputController, DateController, and UserController for handling model-side operations. Observe changes in LedgerManager, NeedsManager, FundingManager, and UserManager, and display updates. Format and print details of Needs, Bundles, Funding Source, and Entries. Handle workflows for updating or deleting items interactively.
Collaborators (implements)	**java.util.Observer** - To register for all FileManager's update notifications.
Collaborators (observes)	**Model.LedgerManager** – Observed for ledger entry changes and updates. **Model.NeedsManager** – Observed for needs and bundles changes. **Model.FundingManager** – Observed for funding sources changes. **Model.UserManager** – Observed for user changes.
Collaborators (uses)	**java.util.Scanner** – Collect input from the console. **Controller.InputController** - Execute need, bundle, funding source, and entry operations requested by the user. **Controller.DateController** - Execute date operations requested by the user. **Controller.UserController** - Execute user operations requested by the user.

Class SwingUI	
Responsibilities	Route user commands to InputController, DateController, and UserController for handling model-side operations. Observe changes in LedgerManager, NeedsManager, FundingManager, and UserManager, and display updates. Format and print details of Needs, Bundles, Funding Source, and Entries. Handle workflows for updating or deleting items interactively. Utilize reusable UI components to render the window.
Collaborators (implements)	**java.util.Observer** - To register for all FileManager's update notifications.
Collaborators (inherits)	**javax.swing.JFrame** - Display the main application window.
Collaborators (observes)	**Model.LedgerManager** – Observed for ledger entry changes and updates. **Model.NeedsManager** – Observed for needs and bundles changes. **Model.FundingManager** – Observed for funding sources changes. **Model.UserManager** – Observed for user changes.
Collaborators (uses)	**java.awt.event.ActionEvent** - Handle action events, such as button clicks, and interact with the controllers to send data. **View.Components.*** - **Reusable UI components** - These are classes that contain static create methods which instantiate UI components. They will **NOT** be listed in this section, as they are implementation details without independent responsibilities in the system; however, the following table lists all available UI creation methods.

	Controller.InputController - Execute need, bundle, funding source, entry operations requested by the user. **Controller.DateController** - Execute date operations requested by the user. **Controller.UserController** - Execute user operations requested by the user.

View.Components.*	
Responsibilities	Hold a collection of public static helper methods that create the reusable UI component.
Static Help Methods	JButton createButton(String text, ActionListener action) JButton createSecondaryButton(String text, ActionListener action) JButton makeFlatButton(String text) JLabel createIcon(String path) JPanel createInput(String label) JPanel createVSeperator() JPanel createHSeperator() JPanel createEditListRow(String item, RowEditProvider actions) JPanel createCounterListRow(String item, Counter counter) JPanel createCheckboxListRow(String item) JPanel createRow(String item); void openAddBundleDialog(JFrame parent, InputController inputCtrl, NeedsManager needsMgr) void openEditBundleDialog(JFrame parent, InputController inputCtrl, NeedsManager needsMgr, Bundle bundleToEdit) void openBundleDialog(JFrame parent, InputController inputCtrl, NeedsManager needsMgr, Bundle existingBundle) void openAddBundlesFulfilledDialog(JFrame parent, InputController inputCtrl, NeedsManager needsMgr, LedgerManager ledgerMgr) void openAddFundingDialog(JFrame parent, InputController inputCtrl) void openEditFundingDialog(JFrame parent, InputController inputCtrl, FundingSource sourceToEdit) void openFundingDialog(JFrame parent, InputController inputCtrl, FundingSource existingSource) void openAddFundingReceivedDialog(JFrame parent, InputController inputCtrl, FundingManager fundingMgr, LedgerManager ledgerMgr) void openAddNeedDialog(JFrame parent, InputController inputCtrl) void openEditNeedDialog(JFrame parent, InputController inputCtrl, BasicNeed needToEdit) void openNeedDialog(JFrame parent, InputController inputCtrl, BasicNeed existingNeed) void openAddNeedsFulfilledDialog(JFrame parent, InputController inputCtrl, NeedsManager needsMgr, LedgerManager ledgerMgr) void openSignOutDialog(JFrame parent, SignOutCallback callback) void openDateSelectorDialog(JFrame parent, DateController dateCtrl, LedgerManager ledgerMgr, Runnable onSuccess) boolean openDeleteDialog(JFrame parent, String itemType, String itemName, String warningMessage)

	boolean openDeleteDialog(JFrame parent, String itemType, String itemName) boolean openDeleteDialog(JFrame parent, String itemType) JPanel createListPanel(List<String> items, RowFactory factory, int height) void showError(String message) void openLoginDialog(JFrame parent, UserController userCtrl) void openSetAvailableFundsDialog(JFrame parent, InputController inputCtrl, LedgerManager ledgerMgr) void openSetThresholdDialog(JFrame parent, InputController inputCtrl, LedgerManager ledgerMgr) void showSuccess(String message) void openSignUpDialog(JFrame parent, UserController userCreate) void onEdit(String item) void onDelete(String item) JPanel createRow(String item) Component spacerMD() Component spacerSM() JLabel createH1(String title) JLabel createH2(String title) JLabel createH3(String title) JLabel createP(String title)
Collaborators (uses)	**javax.swing.JPanel** - Container that holds and organizes other components using a layout manager. **javax.swing.JButton** - A clickable button component that can trigger actions. **javax.swing.JLabel** - Display a short string or an image icon, often used as a label for other components. **java.awt.Dimension** - Represent the width and height of a component. **java.awt.Color** - Represent colors for UI components. **java.awt.Font** - Represent font type, style, and size for text in components. **javax.swing.ImageIcon** - Hold an image that can be displayed in labels, buttons, etc. **javax.swing.JTextField** - A single-line text input field for user input. **javax.swing.BoxLayout** - Arrange components either vertically or horizontally in a container. **java.awt.FlowLayout** - A layout manager that arranges components in a left-to-right flow. **javax.swing.JScrollPane** - Provide a scrollable view for a component. **java.awt.BorderLayout** - Arranges components in five regions: north, south, east, west, center. **javax.swing.JCheckBox** - A checkbox component that can be selected or deselected. **javax.swing.JDialog** - A top-level window used to display a modal or modeless dialog.

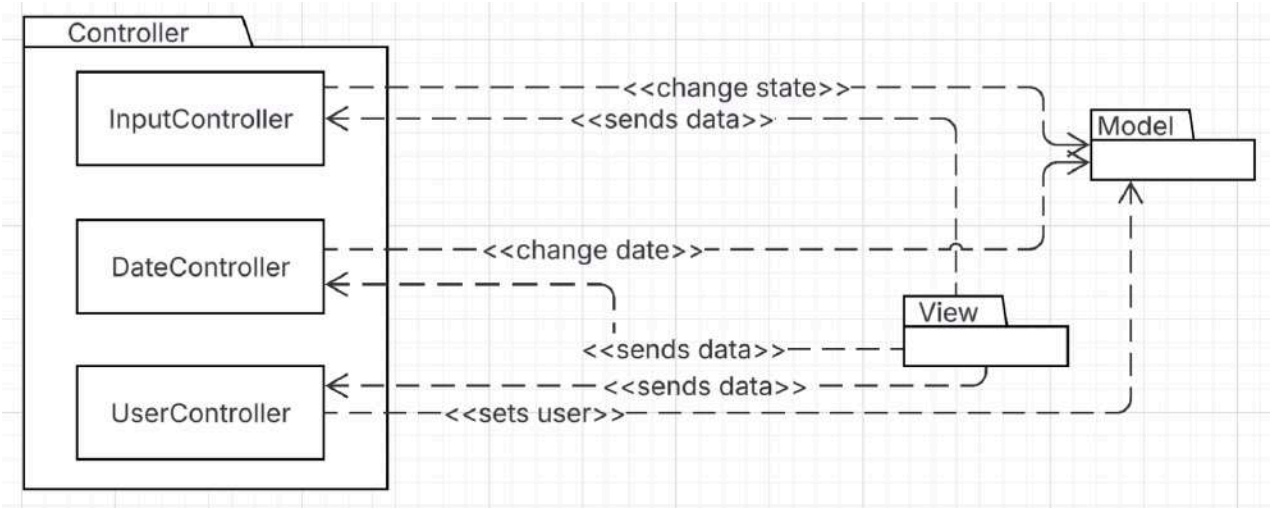

View
JFrame
<<interface>>
Observer
<<observer>>
Model
SwingUI
CLI
<<uses input>>
Controller
Components

4.4 Controller Subsystem

Class InputController	
Responsibilities	Receive operation requests from the UI (e.g., create, read, update, delete). Call the appropriate methods on model classes (LedgerManager, NeedsManager, etc.). Handle data transformation between UI input and model objects.
Collaborators (uses)	**Model.LedgerManager** – For managing entries. **Model.NeedsManager** – For managing needs and bundles. **Model.FundingManager** – For managing funding sources.
Collaborators (receives data from)	**View.CLI** – Receive commands from the user interface. **View.SwingUI** – Receive commands from the user interface.

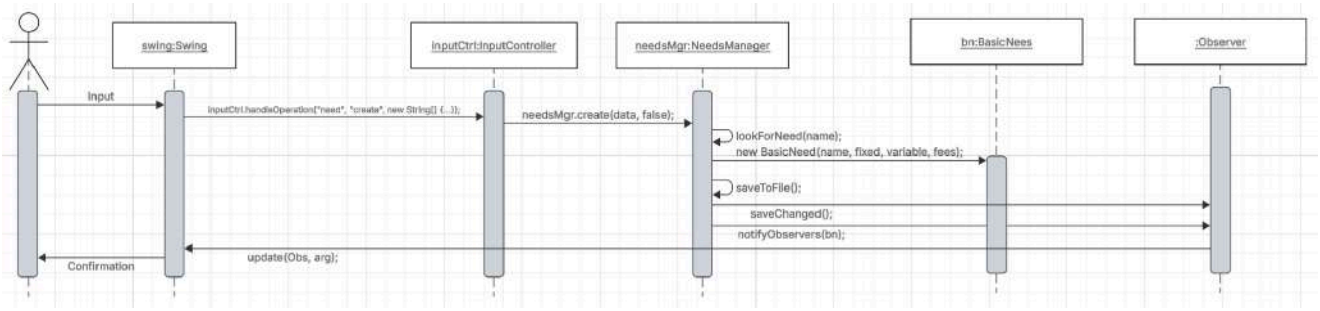
Class DateController	
Responsibilities	Process date selection input from the UI.
Collaborators (uses)	**Model.LedgerManager** – for managing entry dates.
Collaborators (receives data from)	**View.CLI** – Receive commands from the user interface. **View.SwingUI** – Receive commands from the user interface.

Class UserController	
Responsibilities	Process user login input from the UI.
Collaborators (uses)	**Model.UserManager** - **Model.LedgerManager** – For setting the user's ledger file. **Model.NeedsManager** –For setting the user's need file. **Model.FundingManager** – For setting the user's funding source file.
Collaborators (receives data from)	**View.CLI** – Receive commands from the user interface. **View.SwingUI** – Receive commands from the user interface.


Controller
InputController
DateController
UserController
<<change state>>
<<sends data>>
<<change date>>
<<sends data>>
<<sends data>>
<<sets user>>
Model
View

5 Sequence Diagrams

5.1 Sequence Diagram #1 – Creating need



Purpose

This sequence diagram shows how the system handles creating a new Need when a user requests it.

It focuses on passing user input through the CLI/Swing, into InputController and NeedsManager, and then constructing and saving a new BasicNeed while notifying observers of new data.

Scenario and Feature

A user enters a command to create a new need (e.g., “create need food total 10”).

The system validates the input, creates a new BasicNeed, stores it, and updates observers so all interfaces refresh automatically.

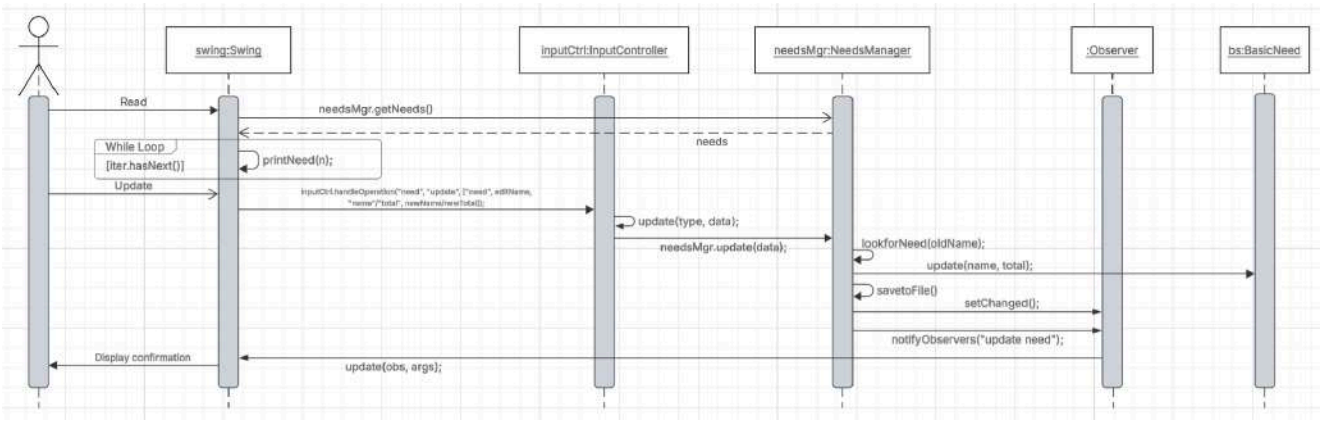
Process Explanation

1. **User -> Swing/CLI**
   - The user enters the creation command.
   - CLI captures this raw input.
2. **Swing/CLI -> InputController**
   - Swing calls handleOperation("Need", "create", new String[]{...}).
3. **InputController -> NeedsManager**
   - InputController parses the input.
   - Calls create(data, false) on NeedsManager.
4. **NeedsManager -> BasicNeed**
   - NeedsManager checks if a need with the same name already exists using lookForNeed(name).
   - If it does not exist:
     - A new BasicNeed(name, total) object is created.
5. **NeedsManager -> Storage**
   - Calls saveToFile() to persist the new need.
6. **NeedsManager -> ObserverPattern**
   - Calls setChanged().

- Calls notifyObservers() to send the new BasicNeed to observers.

7. **Observer -> UI**
    - UI refreshes and displays confirmation to the user.

5.2 Sequence Diagram #2 – Read Needs + Update



Purpose

This sequence diagram shows how the system retrieves all existing needs, displays them to the user, accepts updated values for a selected need, and then updates that need in storage.

It focuses on how the request moves from the UI/CLI, through the InputController to the NeedsManager, and how the Observer pattern ensures that once a need is updated, all observers (UI, logs, etc.) automatically refresh.

The goal is to successfully update a BasicNeed and notify all interested observers.

Scenario and Feature

The user requests to view all needs, selects one need to modify, and enters new values.

The system retrieves all stored needs and displays them.

Once the user submits updated values, the system updates the selected BasicNeed, saves the change to storage, and notifies observers.

All interfaces refresh themselves to show the updated need.

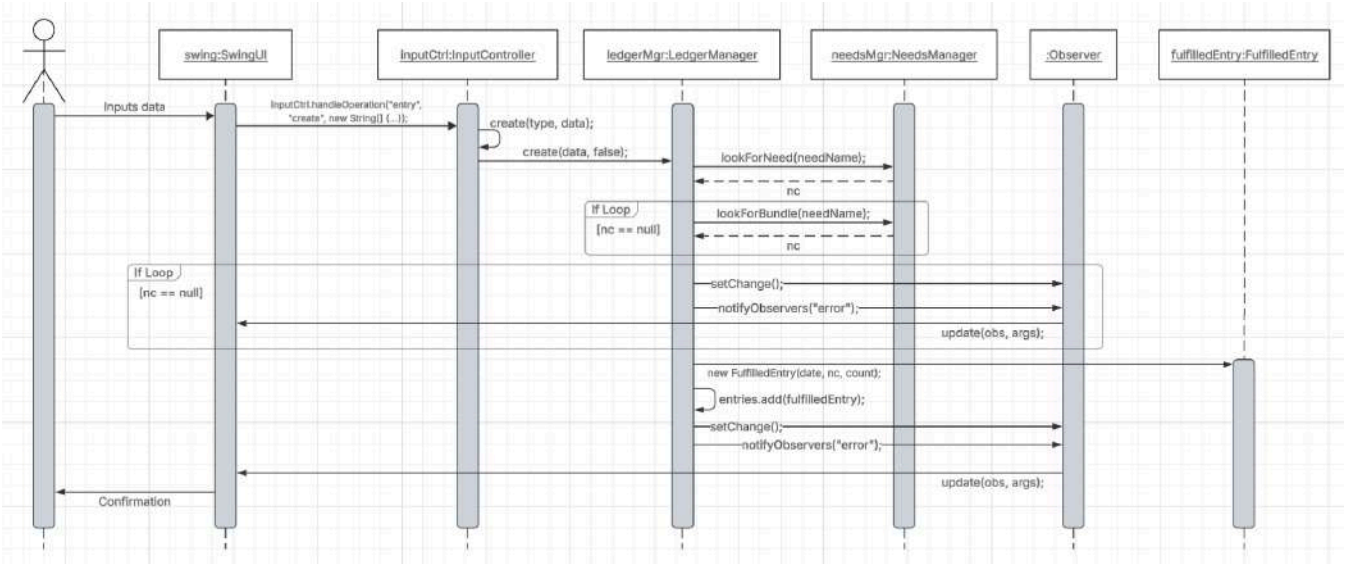
Process Explanation

1. **User -> Swing/CLI**
   - User initiates reading needs (e.g., “read needs”).
   - The user later enters updated values.
2. **Swing/CLI -> NeedManager**
   - Swing calls getNeeds().
   - Display the retrieved needs using a loop.
3. **InputController -> NeedsManager (Update Step)**
   - After the user selects a need and enters new values, InputController calls: update(type, data)

- NeedsManager finds the correct need via lookForNeed(oldName).

4. **NeedsManager -> BasicNeed**
   - The BasicNeed object is modified with the new name and values: update(name, total).
5. **NeedsManager -> Storage**
   - The updated need is written to the file using saveToFile().
6. **NeedsManager -> ObserverPattern**
   - NeedsManager calls setChanged()
   - Then notifyObservers() sends the updated BasicNeed back to observers.
7. **Observer -> UI**
   - Observers refresh the display.
   - The system shows “update successful” to the user.

5.3 Sequence Diagram #3 – Fulfilled Entry



Purpose

This sequence diagram shows how the system handles the creation of a **Fulfilled Entry**, which records that a need has been fulfilled on a specific date and count.

It focuses on how the InputManager interacts with LedgerManager and NeedsManager to validate that the need exists, how errors are handled, and how observers are notified of the new entry or any issues.

Scenario and Feature

A user enters fulfillment data (e.g., “fulfill need food count 3 date 2025-11-01”).

The system checks whether the referenced need or bundle exists.

If valid, it creates a new FulfilledEntry.

If not valid, an error is triggered, and observers are notified.

All observers refresh their interface as needed.

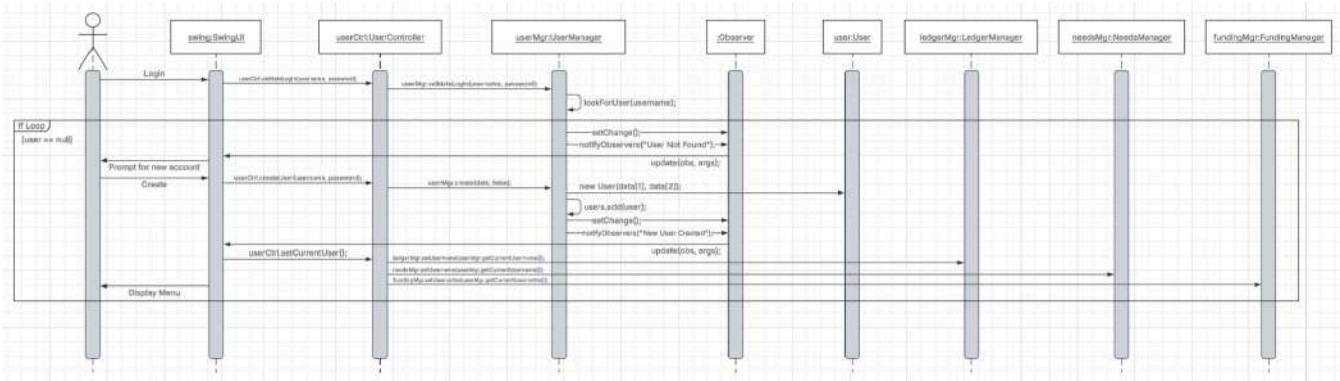
Process Explanation

1. **User -> Swing/CLI**
   - User submits fulfillment input (date, need name, count).
2. **Swing/CLI -> InputController**
   - Swing/CLI forward as handleInput(inputs).
3. **InputController -> LedgerManager**
   - InputController interprets the request and calls create(type, data)

- LedgerManager processes the fulfillment request.

4. **LedgerManager -> NeedsManager**
   - LedgerManager checks whether the need exists via lookForNeed(needName).
   - If null, it checks if it is a bundle using lookForBundle(needName).
5. **NeedsManager -> LedgerManager**
   - If neither a need nor a bundle exists:
     - LedgerManager performs:
       - updateDaily()
       - setChange()
       - notifyObservers("error")
     - UI receives an error message.
6. **LedgerManager -> FulfilledEntry**
   - If the need exists:
     - LedgerManager creates
       new FulfilledEntry(date, nc, count).
7. **LedgerManager -> ObserverPattern**
   - LedgerManager calls setChange().
   - LedgerManager calls notifyObservers().
8. **Observer -> UI**
   - Observers update the UI to show the successful Fulfilled Entry.
   - The system displays confirmation to the user.

5.4 Sequence Diagram #4 – User Login (New Account)



Purpose

This sequence diagram illustrates how the system handles a user attempting to log in, failing because the account does not yet exist, and then successfully creating a new account.

It highlights communication between SwingUI, UserController, UserManager, the Observer interface, and various managers that rely on the current user

Scenario and Feature

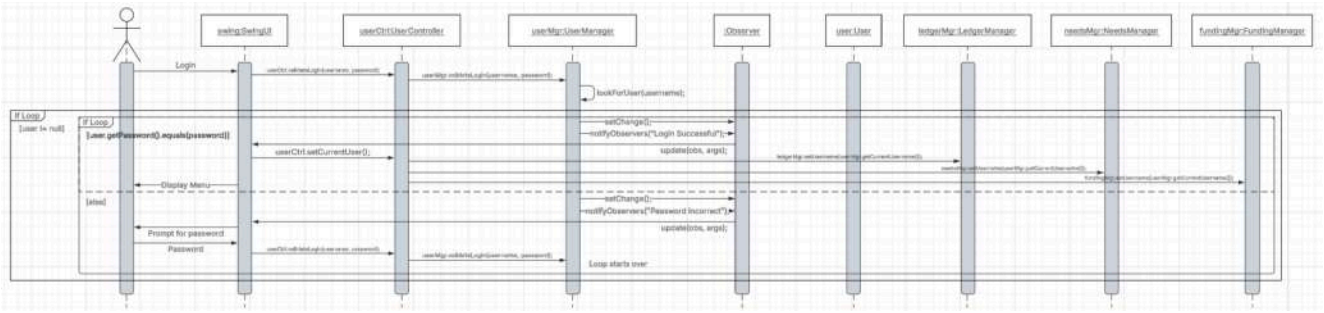
A user tries to log in with a username that does not exist. The system checks the stored users, notifies the UI that the user was not found, and prompts the user to create a new account. After the user enters new account information, the system creates the new user, notifies observers, and initializes the session by updating all managers with the new username.

Process Explanation

1. **User -> SwingUI**
   - User enters login information (username + password).
   - SwingUI captures this input.
2. **SwingUI -> UserController**
   - SwingUI calls validateLogin(username, password).
3. **UserController -> UserManager**
   - UserController forwards the login request by calling validateLogin(username, password).
   - UserManager calls lookForUser(username) to search stored user data.
   - The method returns null because the username does not exist.
4. **UserManager -> ObserverPattern**
   - Since no user is found, UserManager calls notifyObserver("User Not Found").
5. **Observer -> SwingUI**
   - The observer sends the update message to SwingUI. SwingUI displays the “User Not Found” message to the user.
   - Prompts the user to create a new account.

6. **User -> SwingUI**
    - User enters new account information (new username + password).
7. **SwingUI -> UserController**
    - SwingUI calls createUser(username, password).
8. **UserController -> UserManager**
    - UserController forwards the request by calling create(data, false).
9. **UserManager -> User**
    - UserManager creates a new user object using new User(data[1], data[2]).
10. **UserManager**
    - The new user is added to the internal user collection.
11. **UserManager -> ObserverPattern**
    - UserManager calls notifyObserver("New User Created").
12. **Observer -> SwingUI**
    - Observer updates the UI again with the success message.
13. **SwingUI -> UserController**
    - SwingUI receives the update and calls setCurrentUser() on UserController.
14. **UserController -> LedgerManager / NeedsManager / FundingManager**
    - Each manager is updated with the logged-in user via setUsername(userMgr.getCurrentUsername()).
15. **UI -> User**
    - SwingUI transitions the user to the main menu interface.

5.5 Sequence Diagram #5 – User Login (Existing Account)



Purpose

This sequence diagram illustrates how the system handles logging in a returning user using an existing account.

It focuses on validating the username, checking password correctness, updating the current user, and notifying observers of successful or failed login attempts.

The goal is to ensure secure authentication and automatic synchronization of all dependent managers after the user logs in.

Scenario and Feature

A user attempts to log in by providing a username and password.
The system verifies whether the username exists.
If it does, the system checks the provided password against the stored password.

- If the password is correct, the user is logged in, observers are notified of a successful login, and all managers update themselves with the current username.
- If the password is incorrect, an error message is triggered, observers are notified, and the login attempt loops until the user enters the correct password.

This ensures that users cannot access the system without valid credentials and that all internal components know which user is active.

Process Explanation

1. **User -> CLI/SwingUI**
   - The user enters a username.
2. **SwingUI -> UserController**
   - Calls userCtrl.validateLogin
3. **UserController -> UserManager**
   - UserController delegates validation to userManager using validateLogin
4. **UserManager -> UserManager**
   - Looks for the respective username, triggers setChange(), notifyObservers("Password Incorrect"), and notifies the UI about the error. The Login loop restarts

5. **UserManager -> Observer**
    - If the password matches, UserManager performs: setChange() , notifyObservers("Login Successful")
    - Observers notify the UI to advance to the main menu.
6. **UserController -> User**
    - Sets the authenticated user using userCtrl.setCurrentUser()
7. **UserController -> Managers**
    - All managers synchronize using setUsername(userMsgr.getCurrentUsername()) to ensure it logs and processes data to the user it belongs to.
8. **Observer -> UI**
    - Updates the UI to display the main system menu.

6 Pattern Usage

6.1 Pattern #1 - Composite Pattern

The Bundle acts as a composite of BasicNeed components. Both Bundle and BasicNeed implement the NeedComponent interface, so operations like calculating total cost or checking availability can be applied uniformly to a single need or a collection of needs.

Composite Pattern	
Component	NeedComponent
Leaf	BasicNeed
Composite	Bundle
Clients	NeedsManager, LedgerManager

6.2 Pattern #2 - MVC Pattern

User inputs are captured by the Controllers, sent to the Model, and the View updates based on changes in the Model.

MVC Pattern	
Model	FileManager, NeedsManager, LedgerManager, FundingManager, UserManager, Entry, AvailableFundsEntry, ThresholdEntry, FulfilledEntry, FundingSourceEntry, FundingSource, BasicNeed, Bundle, NeedComponent, User
View	CLI, SwingUI
Controller	InputController, DateController, UserController

6.3 Pattern #3 - Observer Pattern

The CLI observes the LedgerManager, NeedsManager, FundingManager, and UserManager, and when the managers notify their subscribers, the CLI/SwingUI updates the display or triggers appropriate actions.

Observer Pattern	
Observer	CLI, SwingUI
Observable	FileManager, LedgerManager, NeedsManager, FundingManager, UserManager

7 Rationale

10/19/25 - Added foundational design, no major changes.

10/24/25 - Redesign the class diagram, mainly adding an InputManager and observer pattern for a better MVC pattern.

11/5/25 - Redid and improved design overview, revised onto subsystems, sequence diagrams, and pattern usage.

11/17/25 - Updated the diagrams to reflect phase 2 requirements, and reflected those changes in the design document.

11/25/25 - Added User Manager and Controller to fulfill the new requirements of the login system

12/1/25 - Updated the diagrams to reflect the latest implementation.

12/3/25 - Reflected the changes in the design document.